

OptiLearn – Feature Documentation

Layer	Tech
AI Model (training)	Gemma 4 9B — Unsloth QLoRA, GCP A100 40GB, r=16, alpha=32, 2 epochs, ~3.5hrs
AI Model (runtime)	Gemma 4 9B — q4_K_M GGUF served via Ollama at localhost:11434
Dev fallback	Gemini 2.0 Flash (USE_LOCAL_OLLAMA=false) — same interface, fast iteration
Speech-to-Text	whisper.cpp — ggml-small.bin, multilingual, 466MB, offline CPU
Text-to-Speech	piper-tts — offline, per-language .onnx voice models (~50MB each)
Embeddings	sentence-transformers paraphrase-multilingual-MiniLM-L12-v2 (420MB)
Vector Search	FAISS — faiss-cpu, IndexFlatL2(384), 10k+ passages
Backend	FastAPI + uvicorn + httpx + aiosqlite + tenacity
Database	SQLite — WAL mode for concurrent students
Frontend	React 18 + Vite + React Router + TanStack Query v5
Offline PWA	vite-plugin-pwa + service worker
Language detection	langdetect (fallback, threshold 0.85)
Image processing	Pillow — resize 1024px max, JPEG encode
PDF processing	PyMuPDF — text extraction from teacher uploads
PDF export	reportlab — teacher reports + note exports
Network	Local WiFi hotspot — teacher laptop as server, no internet

UNIQUE FEATURES

These three features define OptiLearn's differentiation from existing EdTech. They are the core of the hackathon pitch.

Feature 1 — Language-Adaptive AI Tutor

What it does

Student uploads any learning material — textbook photo, PDF, or typed text. OptiLearn detects the source language, translates the entire content into the student's language of choice with meaning preserved, and presents it in a readable format. The student can tap any concept to dive deeper. The AI tutor explains it at the student's specific level using their full learning profile. Voice chat mode available for lower-literacy students.

Differentiator from Google Translate

Google Translate: word-for-word surface translation, no comprehension of the reader, no memory, no follow-up, requires internet, cannot make content interactive.

OptiLearn: translates for understanding — simplifies for grade level, preserves mathematical/scientific meaning, makes content interactive, adapts to student's mastery history, explains concepts Socratically.

Input pipeline

- Photo → Pillow preprocessing (resize 1024px max) → base64 → Gemma 4 multimodal inference
- PDF → PyMuPDF extracts text per page → sent as text to Gemma 4
- Typed text → sent directly to Gemma 4

Translation prompt (sent to Gemma 4)

```
Translate the following educational content into {target_language}.
```

```
The reader is a student in grade {grade_level}, age {age}.
```

```
Rules:
```

- Preserve all mathematical, scientific, and factual meaning exactly
- Simplify vocabulary for grade {grade_level}
- Do not translate proper nouns, formulas, or units
- If a concept has no equivalent, explain it simply
- Output only the translated content, no commentary

After translation

- Full translated text chunked into sections
- Each section embedded with MiniLM-L12-v2
- Added to FAISS index with metadata: {source: 'uploaded_material', student_id, subject}
- Becomes part of student's personal retrieval corpus for future tutoring sessions

Voice chat mode

- Student speaks → whisper.cpp transcribes (offline) → sent to Gemma 4 as text

- Gemma 4 generates response → piper-tts reads aloud → text also displayed on screen simultaneously
- Full conversation history maintained in SQLite messages table

Student profile used in every response

language	ISO 639-1 code — all responses in this language
grade_level	1–8 — vocabulary complexity calibrated to this
mastery_scores	Per-topic float 0.0–1.0 — avoids re-explaining mastered content
level	beginner / intermediate / advanced — depth of explanation
session_history	Topics covered, quiz performance, learning pace

Feature 2 — Live Translator and Note-Taker

What it does

Student opens this feature during a class they cannot understand. The microphone captures the teacher's speech in 5-second chunks. `whisper.cpp` transcribes it offline in the teacher's language. Gemma 4 translates to the student's language and streams text to their screen in near real-time. When the student taps 'End class', all chunks are assembled into a full transcript and Gemma 4 generates structured study notes. Notes are saved, indexed into FAISS, and each paragraph has a speaker button for TTS playback via `piper-tts`.

Full data flow

- MediaRecorder API captures audio in 5-second chunks (WebM format)
- Client-side: WebM blob converted to WAV using `convertToWav()`
- POST `/api/translate/chunk` (multipart WAV + `session_id`)
- `whisper_client.py` calls `whisper.cpp` binary via `asyncio.create_subprocess_exec`
- Returns transcribed text in teacher's language
- FastAPI sends translation prompt to Gemma 4 via Ollama
- SSE stream returns translated text token by token to `LiveTranslator.jsx`
- Each chunk stored in `class_notes.translated_chunks` (JSON array in SQLite)

On 'End class' tap

- POST `/api/translate/end` → background task triggered
- All `translated_chunks` concatenated into `full_transcript`
- Note generation prompt sent to Gemma 4 (non-streaming, full response awaited)
- Notes saved to `class_notes.notes_text` in SQLite
- Notes split into paragraphs → embedded → added to FAISS (source: 'class_note')
- `faiss_indexed` flag set to 1 in `class_notes` table
- `NoteViewer.jsx` renders notes with per-paragraph TTS buttons

Note generation prompt

```
You are creating study notes for {name}, age {age}, grade {grade_level}.
Create structured study notes in {language}:
- Use ## headings for each concept
- Bullet points for key facts
- End with 'In summary:' section (3-5 takeaways)
- Grade-appropriate vocabulary
- Warm, encouraging tone
```

`whisper_client.py` — key implementation

```
async def transcribe_chunk(audio_bytes, hint_language=None):
    # Write WAV to /tmp/optilearn/audio/ (auto-cleaned)
    # Call: whisper.cpp/main -m ggml-small.bin -f chunk.wav --output-txt
    # Returns: transcribed text string
    # Latency: ~3-5 seconds per 5-second chunk on CPU
```

tts_client.py — key implementation

```
VOICE_MAP = {
    'en': 'en_US-lessac-medium',    'ar': 'ar_JO-kareem-medium',
    'fr': 'fr_FR-siwis-medium',    'sw': 'sw_CD-lanfrica-medium',
    'so': 'so_SO-default',          'am': 'am_ET-default',
    'ti': 'en_US-lessac-medium',    # fallback until Tigrinya voice available
}

async def speak(text, language) -> bytes: # returns WAV bytes
```

Feature 3 — Hybrid Offline / Online Functionality

What it does

OptiLearn is offline-first but intelligently uses the internet when available. In fully offline mode, everything runs on the teacher's laptop — Ollama serves Gemma 4 locally, students connect via WiFi hotspot, all data stays on the machine. When internet is detected, the app can route inference to Gemini 2.0 Flash for faster responses on slow hardware. One environment variable controls which mode is active.

Connectivity detection

```
async def check_connectivity() -> bool:
    try:
        async with httpx.AsyncClient(timeout=3.0) as c:
            await c.get('https://dns.google')
            return True
    except Exception:
        return False
```

Inference routing (ollama_client.py)

```
async def get_client():
    if os.getenv('USE_LOCAL_OLLAMA', 'true').lower() == 'false':
        return GeminiClient()    # google-generativeai SDK
    return OllamaClient()        # httpx → localhost:11434
```

Both clients expose the same interface: `chat(messages, tools, stream)`. The rest of the backend never knows which is running.

.env flags

```
USE_LOCAL_OLLAMA=true      # classroom deployment (no internet)
USE_LOCAL_OLLAMA=false     # development (routes to Gemini 2.0 Flash)
OLLAMA_MODEL=optilearn-gemma4
CLOUD_ENDPOINT=https://generativelanguage.googleapis.com/v1beta
```

Classroom network architecture

- Teacher's laptop broadcasts WiFi hotspot (no internet)
- Hotspot IP: 192.168.137.1 (Windows) or 192.168.0.1 (Mac/Linux)
- Students open `http://{hotspot_ip}:8000` in any browser
- PWA service worker installs app locally on first visit
- All student data written to SQLite on teacher's laptop in real time
- Teacher dashboard reads from same DB — no sync step needed
- SQLite WAL mode handles concurrent writes from 30+ students safely
- React Query `refetchInterval`: 10 seconds for teacher dashboard updates

LMS FEATURES

These four features form the Learning Management System core. They work together as an integrated platform — quiz results feed the progress tracker, which feeds the teacher dashboard, which uses curriculum retrieval to ground all AI responses.

LMS Feature 1 — Teacher Dashboard

What it does

Full LMS teacher interface. Upload textbooks and lesson slides for students. Create and assign quizzes manually. View individual student progress with AI-generated reports. Monitor the entire class via mastery heatmap. Download weekly PDF reports. Alerts for inactive students and stuck topics.

Sub-features

Material upload

- Teacher uploads PDFs, images, or slide decks via the dashboard
- Files stored in `data/materials/` directory
- PyMuPDF extracts text from PDFs
- `POST /api/materials/upload` → extract text → embed → index into FAISS
- Immediately available to all students via `curriculum_retrieval` tool
- Stored in materials SQLite table with `title`, `subject`, `file_path`, `faiss_indexed` flag

Quiz builder

- Teacher creates multiple-choice or short-answer questions manually
- Saved to `teacher_quizzes` table with `title`, `subject`, `questions` (JSON array), `assigned_to`
- `assigned_to`: specific `student_id` or 'all' for whole class
- Students see teacher-assigned quizzes in their session alongside AI-generated ones

Student progress view

- Per-student page: mastery heatmap, session history, quiz scores over time
- AI-generated progress report — streamed via SSE from `GET /api/students/{id}/report`
- Report generated by Gemma 4 using full mastery history + last 10 quiz results

AI progress report prompt

```
Generate a 3-paragraph progress report for teacher use.
Be specific: what student mastered, what they struggle with,
one concrete recommendation for tomorrow's lesson.
Student: {name}, age {age}, grade {grade_level}, language {language}
Mastery: {topic_mastery_dict}
Recent quizzes: {last_10_results}
```

Class overview

- Grid of all students — level badge, last active, mastery average, 5-quiz sparkline

- Sorted by 'needs attention' — inactive or stuck students float to top
- Alerts: inactive_3_days, stuck_on_topic, level_dropped
- React Query refetchInterval: 10s — GET /api/teacher/students

New SQLite tables

```
CREATE TABLE materials (  
  id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(8)))),  
  title TEXT NOT NULL, subject TEXT, file_path TEXT NOT NULL,  
  uploaded_by TEXT, faiss_indexed INTEGER DEFAULT 0,  
  created_at TEXT DEFAULT (datetime('now'))  
);  
  
CREATE TABLE teacher_quizzes (  
  id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(8)))),  
  title TEXT NOT NULL, subject TEXT,  
  questions TEXT NOT NULL, -- JSON array  
  assigned_to TEXT DEFAULT 'all',  
  created_at TEXT DEFAULT (datetime('now'))  
);
```


LMS Feature 2 — Curriculum Retrieval

What it does

Semantic FAISS search over an offline vector store of pre-embedded curriculum content. Gemma 4 calls this tool automatically before explaining any concept, grounding its response in vetted educational content. Returns top 3 most relevant passages with source attribution. Teacher-uploaded materials and student class notes are part of the same index.

Tool definition (passed to Gemma 4 in every request)

```
{
  'name': 'curriculum_retrieval',
  'description': 'Search offline curriculum before explaining any concept.'
                'ALWAYS call this before explaining something new.',
  'parameters': {
    'topic': string,          // e.g. 'fractions', 'photosynthesis'
    'grade_level': integer,  // 1-8
    'language': string       // ISO 639-1
  }
}
```

FAISS store implementation (faiss_store.py)

```
class CurriculumStore:
    def query(self, topic, grade_level, language, k=3):
        query = f'grade {grade_level} {topic} explanation'
        embedding = self.model.encode([query], normalize_embeddings=True)
        distances, indices = self.index.search(embedding, k)
        return [self.meta[str(idx)] for idx in indices[0] if idx != -1]

    def add_passages(self, passages): # called after notes/material upload
        embeddings = self.model.encode([p['text'] for p in passages])
        self.index.add(embeddings)
        self.index.write_index(INDEX_PATH)
```

Corpus content

Subject	Grade
Math	1–6
Science	3–6
Literacy	1–6
Life Skills	All
Teacher uploads	All
Student notes	All

build_index.py flow

- Read all .txt files from data/curriculum/ recursively
- Split into passages of ~200 words with 50-word overlap
- Embed with paraphrase-multilingual-MiniLM-L12-v2
- Build IndexFlatL2(384) FAISS index
- Save curriculum.index + curriculum_meta.json
- Runtime: ~3 minutes for 7,500 passages on CPU

LMS Feature 3 — Quiz Generator

What it does

Generates adaptive multiple-choice questions on a topic after Gemma 4 has explained a concept. Calibrated to student's current level (beginner/intermediate/advanced), in student's language. One question at a time. Level adapts automatically. Trauma-aware UI — never uses 'wrong', 'incorrect', or 'failed'. Teacher-created quizzes also surfaced here.

Tool definition

```
{
  'name': 'quiz_generator',
  'parameters': {
    'topic': string,
    'level': 'beginner' | 'intermediate' | 'advanced',
    'language': string, // ISO 639-1
    'n': integer // default 3
  }
}
```

Quiz generation prompt

```
Generate {n} quiz questions about '{topic}' for a {level} student.
Language: {language}. Return ONLY a JSON array:
[
  {
    'question': '...',
    'type': 'multiple_choice',
    'options': ['A...', 'B...', 'C...', 'D...'],
    'correct': 'A',
    'explanation': '...'
  }
]
```

Level adaptation logic

```
def compute_new_level(current_level, recent_scores):
    if len(recent_scores) < 3: return current_level
    avg = sum(recent_scores[-3:]) / 3
    if avg >= 0.85 and current_level == 'beginner': return 'intermediate'
    if avg >= 0.85 and current_level == 'intermediate': return 'advanced'
```

```

if avg <= 0.40 and current_level == 'advanced': return 'intermediate'
if avg <= 0.40 and current_level == 'intermediate': return 'beginner'
return current_level

```

Trauma-aware UI rules (non-negotiable)

- Never use: 'wrong', 'incorrect', 'mistake', 'failed'
- Wrong answer: 'Let's look at this again together.' → show explanation → re-explain with different approach
- Correct answer: specific, enthusiastic praise ('Yes! The bottom number is called the denominator!')
- One question at a time — never show all at once
- Progress dots at bottom — no percentage displayed (avoids discouragement)

LMS Feature 4 — Progress Tracker

What it does

Records every quiz result and updates topic mastery using an exponential moving average. Determines level changes from last 3 quiz averages. Feeds the teacher dashboard directly — no separate analytics pipeline. Teacher sees every student's mastery per topic in real time via a color-coded heatmap. Alerts when students are inactive or stuck.

Tool definition

```

{
  'name': 'update_progress',
  'parameters': {
    'student_id': string,
    'topic': string,
    'score': number,          // 0.0-1.0
    'question_ids': array    // list of answered question IDs
  }
}

```

EMA mastery update

```

new_mastery = 0.7 × old_mastery + 0.3 × new_score
# Upsert into topic_mastery table:
INSERT INTO topic_mastery (student_id, topic, mastery, level, last_updated)
VALUES (?, ?, ?, ?, datetime('now'))
ON CONFLICT(student_id, topic) DO UPDATE SET
  mastery=excluded.mastery, level=excluded.level,
  last_updated=excluded.last_updated

```

Heatmap color coding

Mastery range	Color	Meaning
0.0 – 0.39	Red #E24B4A	Struggling — teacher should prioritize
0.40 – 0.74	Amber #EF9F27	Making progress
0.75 – 1.0	Green #639922	Mastered
No data	Grey #AAAAAA	Not yet attempted

TRAINING PIPELINE

Fine-Tuning — Unsloth QLoRA on GCP A100

Infrastructure

GCP a2-highgpu-1g instance (A100 40GB VRAM). Cost: $\sim \$3.67/\text{hr} \times 3.5\text{hrs} = \sim \13 total. Well within \$300 GCP free credits. Dataset prep: ~ 20 min local. Training: ~ 3.5 hrs on GCP. GGUF conversion + Ollama registration: ~ 30 min local after download.

Training dataset

Source	Volume
Khan Academy transcripts	8,000 examples
UNHCR pedagogy guidelines	500 examples
Multilingual QA (synthetic)	3,000 examples
Combined (combined_train)	11,500+
Eval set (eval_set.jsonl)	60 examples

QLoRA configuration

Parameter	Value
Base model	google/gemma-4-9b-it
LoRA rank (r)	16
LoRA alpha	32
Target modules	q,k,v,o,gate,up,down_proj
Quantization	4-bit NF4 (bitsandbytes)
Max seq length	2048 tokens
Epochs	2
Batch size (effective)	16
Learning rate	2e-4
Runner	train.sh shell script

Evaluation — what gets published

- 60 held-out scenarios scored 1–5 on three dimensions:
 - Socratic quality — does it guide rather than answer directly?
 - Language compliance — is response in the correct language?
 - Trauma-aware tone — does it avoid shame, stay patient?
- Run both base Gemma 4 and fine-tuned OptiLearn on same 60 prompts
- Publish delta table to HuggingFace as benchmark_report.md

- This is required for the Unsloth Special Technology Prize (\$10k)

GGUF conversion + Ollama registration

```
# After training completes on GCP:
python llama.cpp/convert_hf_to_gguf.py ./optilearn-weights \
    --outfile models/optilearn-gemma4-q4_K_M.gguf --outtype q4_K_M

# Register with Ollama:
ollama create optilearn-gemma4 -f Modelfile

# Flip .env flags:
USE_LOCAL_OLLAMA=true
OLLAMA_MODEL=optilearn-gemma4
```

COMPLETE API ROUTE REFERENCE

Method + Route	Description
GET /api/health	Ollama status, model name, DB ok, FAISS passage count
GET /api/students	List all students
POST /api/students	Create student — body: {name, age, language, grade_level}
GET /api/students/{id}	Get single student
PUT /api/students/{id}	Update student fields
GET /api/students/{id}/progress	Mastery scores + recent quizzes
GET /api/students/{id}/report	SSE — AI-generated progress report stream
POST /api/sessions	Create session — body: {student_id, type}
POST /api/chat	SSE stream — body: {session_id, message, image_b64?}
GET /api/sessions/{id}/history	Full message history for a session
POST /api/translate/start	Start live translation — body: {student_id, target_language}
POST /api/translate/chunk	Multipart WAV chunk — SSE stream of translation
POST /api/translate/end	End class → trigger note generation (background task)
GET /api/translate/{id}/notes	SSE stream of note generation progress
GET /api/notes/{id}	Get completed class notes
GET /api/notes/{id}/export	PDF download of notes
POST /api/tts/speak	body: {text, language} → WAV bytes
POST /api/materials/upload	Teacher uploads PDF/image → extract → index into FAISS
GET /api/materials	List all uploaded materials
POST /api/teacher/quiz	Teacher creates manual quiz
GET /api/teacher/quiz	List all teacher-created quizzes
GET /api/teacher/students	Dashboard student list with mastery averages
GET /api/teacher/heatmap	{topics[], students[], grid[][]} — mastery grid
GET /api/teacher/alerts	Inactive + stuck student alerts
GET /api/teacher/report	PDF weekly class report — query: ?week=YYYY-WW
POST /api/quiz/generate	body: {student_id, topic, n?} → quiz questions
POST /api/quiz/submit	body: {student_id, session_id, results[]} → mastery update
GET /{full_path}	SPA fallback — serves React frontend/dist/index.html

DATABASE SCHEMA

Table	Key columns
students	id, name, age, language, grade_level, created_at, last_active
sessions	id, student_id, type (tutoring live_class), started_at, ended_at, message_count, topics_covered (JSON)
messages	id, session_id, role (user assistant tool), content, tool_name, timestamp
quiz_results	id, student_id, session_id, topic, question_text, student_answer, correct_answer, correct (0 1), score, timestamp
topic_mastery	student_id + topic (PK), mastery (0.0–1.0), level (beginner intermediate advanced), last_updated
class_notes	id, student_id, session_id, raw_transcript, translated_chunks (JSON), notes_text, notes_language, subject, faiss_indexed (0 1)
materials	id, title, subject, file_path, uploaded_by, faiss_indexed, created_at
teacher_quizzes	id, title, subject, questions (JSON), assigned_to, created_at

SUBMISSION CHECKLIST

☐	Kaggle writeup — ~1,400 words, under 1,500 word limit
☐	YouTube video — 3 minutes, public, no login required
☐	GitHub repo — public, documented README, well-structured code
☐	Live demo URL — HuggingFace Space or VPS
☐	HuggingFace weights published — fine-tuned optilearn-gemma4
☐	benchmark_report.md published alongside weights on HuggingFace
☐	Cover image attached to Kaggle writeup
☐	Track selected in Kaggle submission form (Future of Education)
☐	Show airplane mode / WiFi disconnected at start of demo video
☐	Submit by May 16 (2 days before May 18 deadline)